

SVERK Rover

Полная техническая документация по текущему ROS 2 workspace

Поле	Значение
Платформа	4-колесный mecanum rover на Raspberry Pi 5
ROS	ROS 2 Jazzy
Основной workspace	~/sverk_rover
Дата документа	12 июля 2026
Назначение	Справочник для хакатона, разработки, диагностики и запуска робота

Документ сформирован из файлов текущего репозитория. Он описывает реализованные ROS-пакеты, launch-файлы, конфиги, веб-интерфейс, экран, периферию, навигацию, карты, прошивку аудиомодуля и типовые команды запуска.

Оглавление

1. Краткая сводка
2. Архитектура workspace
3. Запуск и режимы работы
4. Пакеты ROS 2
5. Аппаратные подсистемы и периферия
6. Топики, сервисы и интерфейсы
7. Веб-интерфейс и экран Raspberry Pi
8. Камера, распознавание объектов и карты
9. Навигация, SLAM и локализация
10. Голосовой модуль Waveshare
11. Конфиги и важные файлы
12. Проверка, диагностика и типовые проблемы

1. Краткая сводка

SVERK Rover - учебно-хакатонная робототехническая платформа на Raspberry Pi 5 с ROS 2 Jazzy. В текущем workspace собраны базовый драйвер шасси, одометрия, локализация, навигация, SLAM, камера, детекция объектов, лидар, Octoliner, светодиодная лента, голосовой ввод, веб-интерфейс, экранное приложение и диагностические средства.

Основные возможности

- Запуск всего робота через `rover_bringup` с разделением на `hardware`, `peripherals` и `ui launch`-файлы.
- Управление месамит-базой через `/cmd_vel` и драйвер моторной платы.
- Одометрия колес, EKF-локализация и Nav2/SLAM Toolbox для карты и навигации.
- USB-камера с JPEG/MJPEG, `get_frame` сервисом и отдельной нодой детекции объектов.
- Веб-интерфейс для мониторинга ROS graph, управления движением, камеры, лидара, света, Octoliner, голосового распознавания, терминала, карт и hackathon-файлов.
- Экран Raspberry Pi с IP/hostname, статусом, технической информацией, Wi-Fi настройками и полноэкранный консолью.
- Поддержка Waveshare ESP32-S3 Audio Board как аудиоисточника для Whisper-распознавания речи.

Ключевые порты и страницы

Компонент	По умолчанию	Назначение
rover_web	http://<ip>:8765	Основная веб-морда робота
ttyd	http://<ip>:7681	Веб-терминал робота
rosboard	http://<ip>:8888	Отдельная ROSboard-страница для просмотра ROS graph и сообщений

2. Архитектура workspace

После реорганизации проект разделен на смысловые области: базовые ROS-пакеты, периферия, UI, модели, карты, firmware и hackathon-файлы. Это помогает не смешивать драйверы железа, веб-интерфейс, конфиги запуска и данные соревнований.

```
sverk_rover/  
  src/  
    rover_bringup/      # launch/config точка сборки системы  
    rover_interfaces/   # custom msg/srv  
    rover_description/  # URDF/Xacro и модель робота  
    rover_navigation/   # Nav2, SLAM, карты/маршруты  
    rover_localization/ # EKF конфиги  
    rover_teleop/       # keyboard/мух управление  
    rover_wheel_odometry/ # одометрия колес  
    rover_vision/       # обработка камеры и нейросети  
    peripherals/        # драйверы внешних модулей  
    ui/                 # веб-интерфейс, экран, rosboard  
  maps/                 # карты Nav2/web-визуализации  
  firmware/             # прошивки внешних модулей  
  hackathon_files/      # регламенты, справочные файлы, этот PDF
```

Основные launch-файлы

Файл	Что запускает
src/peripherals/rover_led_strip/launch/led_strip.launch.py	Launch-файл пакета.
src/peripherals/rover_octoliner/launch/octoliner.launch.py	Launch-файл пакета.
src/peripherals/rover_waveshare_audio/launch/waveshare_audio.launch.py	Launch-файл пакета.
src/rover_bringup/launch/hardware.launch.py	Базовое железо: motor controller/base driver, IMU, lidar и связанные аппаратные ноды.
src/rover_bringup/launch/mapping.launch.py	SLAM Toolbox для построения карты.
src/rover_bringup/launch/navigation.launch.py	Nav2-навигация.
src/rover_bringup/launch/peripherals.launch.py	Камера, vision detector, LED strip, Octoliner, Waveshare audio и другая периферия.
src/rover_bringup/launch/robot.launch.py	Главный запуск робота: включает аппаратную часть, периферию, UI и остальные системные ноды.
src/rover_bringup/launch/ui.launch.py	Веб, ttyd terminal, rosboard, экранное приложение.
src/rover_bringup/launch/update_map.launch.py	Обновление/сохранение карты.
src/rover_description/launch/description.launch.py	Launch-файл пакета.
src/rover_description/launch/display_lidar.launch.py	Launch-файл пакета.
src/rover_description/launch/display_model.launch.py	Launch-файл пакета.
src/rover_description/launch/display_navigation.launch.py	Launch-файл пакета.
src/rover_description/launch/display_odom.launch.py	Launch-файл пакета.
src/rover_description/launch/display_slam.launch.py	Launch-файл пакета.
src/rover_localization/launch/localization.launch.py	Launch-файл пакета.
src/rover_navigation/launch/navigation.launch.py	Nav2-навигация.
src/rover_navigation/launch/slam.launch.py	Launch-файл пакета.
src/rover_navigation/launch/update_map.launch.py	Обновление/сохранение карты.
src/rover_vision/launch/vision.launch.py	Launch-файл пакета.
src/ui/rosboard/launch/rosboard.launch.py	Launch-файл пакета.

Файл	Что запускает
src/ui/rover_display/launch/display.launch.py	Launch-файл пакета.
src/ui/rover_web/launch/web.launch.py	Launch-файл пакета.

3. Запуск и режимы работы

Полный запуск

```
cd ~/sverk_rover
source /opt/ros/jazzy/setup.zsh
source install/setup.zsh
ros2 launch rover_bringup robot.launch.py
```

Главный launch разбит на более мелкие файлы. Это уменьшает кашу в одном большом файле и позволяет запускать отдельно UI или периферию, когда не нужен полный робот.

Отдельные режимы

Команда	Назначение
ros2 launch rover_bringup ui.launch.py	Только веб-интерфейс, rosboard, ttyd и экран Raspberry Pi.
ros2 launch rover_bringup peripherals.launch.py	Только периферия: камера, обработка, свет, Octoliner, голосовой модуль.
ros2 launch rover_bringup navigation.launch.py	Nav2-навигация по карте.
ros2 launch rover_bringup mapping.launch.py	SLAM и построение карты.
ros2 run rover_device_manager setup_devices	Создание стабильных путей устройств, если USB-пути изменились.

Сборка

```
cd ~/sverk_rover
PYTHONNOUSERSITE=1 colcon build --symlink-install
source install/setup.zsh
```

PYTHONNOUSERSITE=1 полезен на ровере, потому что пользовательские pip-пакеты могут конфликтовать с системными ROS/OpenCV/Numpy зависимостями.

4. Пакеты ROS 2

В workspace найдено 19 ROS-пакетов. Ниже сводка по package.xml.

Пакет	Версия	Путь	Назначение
rover_base_driver	0.2.0	src/peripherals/rover_base_driver	cmd_vel to four-wheel motor-controller commands and encoder feedback.
rover_camera	0.1.0	src/peripherals/rover_camera	USB camera driver for publishing raw images from the rover.
rover_device_manager	0.4.1	src/peripherals/rover_device_manager	Automatic detection and verification of rover serial devices.
rover_imu	0.2.0	src/peripherals/rover_imu	Native Yahboom 10-axis IMU serial driver and normalization tools.
rover_led_strip	0.1.0	src/peripherals/rover_led_strip	ROS 2 driver and controller for an addressable LED strip on the rover.
rover_octoliner	0.1.0	src/peripherals/rover_octoliner	ROS 2 driver for the Amperka Octoliner line sensor module.
rover_waveshare_audio	0.1.0	src/peripherals/rover_waveshare_audio	ROS 2 Whisper STT bridge for the Waveshare ESP32-S3-AUDIO-Board.
slidar_ros2	1.0.1	src/peripherals/slidar_ros2	The rplidar ros2 package, support rplidar A2/A1 and A3/S1
rover_bringup	0.4.3	src/rover_bringup	Top-level launch and configuration for the complete rover.
rover_description	0.4.2	src/rover_description	URDF/Xacro model, TF frames and RViz views for the rover.
rover_interfaces	0.2.0	src/rover_interfaces	Custom interfaces for rover sensors, actuators and services.
rover_localization	0.2.0	src/rover_localization	robot_localization EKF configuration for wheel odometry and IMU.
rover_navigation	0.4.3	src/rover_navigation	Nav2, AMCL and SLAM Toolbox configuration for the rover.
rover_teleop	0.4.2	src/rover_teleop	Terminal WASD teleoperation for the four-wheel mecanum rover.
rover_vision	0.1.0	src/rover_vision	Camera image processing and YOLO-like object detection for the rover.
rover_wheel_odometry	0.2.0	src/rover_wheel_odometry	Wheel odometry for a four-wheel mecanum rover.
rosboard	1.3.1	src/ui/rosboard	ROSBoard
rover_display	0.1.0	src/ui/rover_display	Fullscreen touchscreen status display for the rover.
rover_web	0.1.0	src/ui/rover_web	Local web interface, web gateway and route executor for the rover.

Классификация пакетов

Группа	Пакеты	Роль
Core/bringup	rover_bringup, rover_interfaces, rover_description	Сборка системы, сообщения/сервисы и описание робота.
Движение	rover_base_driver, rover_wheel_odometry, rover_teleop	Связь с моторной платой, расчет одометрии и ручное управление.
Навигация	rover_navigation, rover_localization	Nav2, SLAM Toolbox, EKF.
Периферия	rover_camera, rover_vision, rover_imu, rover_led_strip, rover_octoliner, rover_waveshare_audio, slidar_ros2	Камера, нейросеть, IMU, лента, датчики линии, голос и лидар.

Группа	Пакеты	Роль
UI	rover_web, rover_display, rosboard	Основная веб-морда, экран Raspberry Pi и ROSboard.

5. Аппаратные подсистемы и периферия

Сводная таблица железа

Модуль	Пакет/нода	Подключение/параметры	ROS-интерфейс
Моторная плата	rover_base_driver / base_driver_node	USB serial, stable path через device manager; /cmd_vel; timeout 0.5s	/wheel/encoders, /wheel/commands, /odom через wheel odometry
IMU Yahboom	rover_imu / yahboom_imu_node	serial 921600, /tmp/rover_devices/imu	/imu/data, /imu/mag, /imu/euler, /imu/valid_frame_count
SLLidar	slidar_ros2 / slidar_node	serial 460800, /tmp/rover_devices/lidar	/scan
USB camera	rover_camera / usb_camera_node	/dev/video0, 1280x720, 30 FPS, MJPEG	/image_raw, /image_raw/compressed, get_frame
Vision detector	rover_vision / camera_detector_node	fixed model from package models, input /image_raw	/image_processed, /image_processed/compressed, /detections
Octoliner	rover_octoliner / octoliner_node	I2C bus 1, address 0x2a (42)	/octoliner/reading, calibration/sensitivity services
LED strip	rover_led_strip / led_strip_node	16 LEDs, SPI bus 1 device 0, WS2812 SPI backend	/led_strip/state, /led_strip/set_effect, /led_strip/set_leds
Waveshare ESP32-S3 Audio	rover_waveshare_audio / waveshare_audio_node	serial /dev/waveshare_audio, 2 Mbps, WAV chunks to Whisper	/voice/text, /waveshare_audio/status

Device manager

rover_device_manager нужен, чтобы USB-устройства не менялись местами после перезагрузки. Он ищет реальные serial/by-path устройства и создает стабильные ссылки в /tmp/rover_devices: например motor_controller, imu, lidar. Если кабель воткнули в другой порт, discovery может подсказать, что нужно обновить конфиг.

LED strip

Текущая реализация ленты использует SPI-драйвер для WS2812-подобной адресной ленты. В конфиге указано 16 светодиодов. При старте нода выполняет self-test: последовательно зажигает светодиоды, затем гасит ленту. В веб-интерфейсе есть preview, покадровое управление каждым LED и сохранение пользовательских кадров/пресетов.

Octoliner

Octoliner - модуль из 8 датчиков линии. Нода публикует значения сенсоров, рассчитанную позицию линии, признак обнаружения линии и служебное состояние. Настройки чувствительности и калибровки вынесены в настройки веб-интерфейса.

6. Топики, сервисы и интерфейсы

Основные топики

Топик	Тип/источник	Назначение
/cmd_vel	geometry_msgs/Twist	Команда скорости для базы.
/wheel/encoders	rover_interfaces/WheelEncoders	Данные энкодеров от моторной платы.
/odom	nav_msgs/Odometry	Одометрия робота для визуализации, Nav2 и web map.
/imu/data	sensor_msgs/Imu	IMU данные.
/scan	sensor_msgs/LaserScan	Лидар для SLAM/Nav2/web visualization.
/image_raw	sensor_msgs/Image	Исходное изображение USB-камеры.
/image_processed	sensor_msgs/Image	Кадр после нейросетевой обработки с рамками.
/detections	std_msgs/String	JSON-описание найденных объектов: class, confidence, bbox.
/led_strip/state	rover_interfaces/LedStripState	Состояние ленты и текущий эффект.
/octoliner/reading	rover_interfaces/OctolinerReading	Значения 8 датчиков линии и derived-состояние.
/voice/text	std_msgs/String	Распознанный Whisper текст.
/agent/text	std_msgs/String	Сообщения LLM/агента для экранного приложения.

Custom messages

Файл	Назначение
src/rover_interfaces/msg/LEDState.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/msg/LEDStateArray.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/msg/LedStripState.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/msg/OctolinerReading.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/msg/WheelCommand.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/msg/WheelEncoders.msg	Custom интерфейс rover_interfaces.
src/rover_interfaces/srv/GetFrame.srv	Custom интерфейс rover_interfaces.
src/rover_interfaces/srv/SetLEDEffect.srv	Custom интерфейс rover_interfaces.
src/rover_interfaces/srv/SetLEDs.srv	Custom интерфейс rover_interfaces.
src/rover_interfaces/srv/SetLedStripState.srv	Custom интерфейс rover_interfaces.
src/rover_interfaces/srv/SetSensitivity.srv	Custom интерфейс rover_interfaces.

Сервисы

Сервис/тип	Назначение
rover_interfaces/srv/GetFrame	Получить один кадр с камеры без подписки на поток.
rover_interfaces/srv/SetLEDEffect	Задать эффект LED ленты.
rover_interfaces/srv/SetLEDs	Управление отдельными LED.
rover_interfaces/srv/SetLedStripState	Установка состояния LED ленты.
rover_interfaces/srv/SetSensitivity	Настройка чувствительности Octoliner.
std_srvs/Trigger для Octoliner	Калибровка и оптимизация на белом/черном поле.

7. Веб-интерфейс и экран Raspberry Pi

Веб-интерфейс rover_web

Основная веб-морда запускается как ROS-нода `web_gateway_node` и слушает `0.0.0.0:8765`. Она общается с ROS graph напрямую через `roslru`, а также отдает статические страницы, `hackathon`-файлы, карты, терминал и интеграции.

Страница	Функциональность
Главная	Сводная информация, быстрые кнопки ROSboard/terminal и общее состояние робота.
ROS State	Топики, сервисы, ноды; просмотр сообщений, публикация в топики, вызов сервисов.
Движение	WASD управление, маршруты, визуализация одометрии и карты.
Периферия	Камера, лидар, свет, Octoliner, распознавание голоса, сервоприводы при включении в настройках.
Терминал	Встроенный <code>tttyd</code> <code>iframe</code> и кнопка перезапуска терминала.
Диагностика	Системная загрузка, ROS counters, позиция/скорость, диагностика устройств.
Настройки	Настройки камеры, лидара, LED, Octoliner, видимость сервоприводов и прочие параметры.
Хакатон	Просмотр <code>md/html/pdf</code> файлов из <code>hackathon_files</code> .

Экран Raspberry Pi

`rover_display` - локальное экранное приложение. Сейчас основной экран открывается сразу при запуске: сверху компактная панель с `sverk_rover_<serial>` и IP-адресом, по нажатию на IP открываются настройки Wi-Fi. Левая часть показывает техническое состояние, нижняя кнопка открывает полноэкрannую консоль. Правая часть выбирается параметром в `launch/config`: заглушка или `hackathon`-вид с батареей и текстом агента из `/agent/text`.

Параметр	Значение/роль
<code>wifi_interface</code>	<code>wlan0</code>
<code>netplan_path</code>	<code>/etc/netplan/50-cloud-init.yaml</code>
<code>agent_text_topic</code>	<code>/agent/text</code>
<code>battery_topic</code>	<code>/battery/state</code>
<code>console_shell</code>	<code>/bin/bash</code>

8. Камера, распознавание объектов и карты

Камера

Параметр	Значение
Устройство	/dev/video0
Разрешение	1280x720 @ 30 FPS
Топики	/image_raw, /image_raw/compressed
Сервис кадра	get_frame
Формат	MJPEG=True, JPEG quality=85

Нейросетевая обработка

rover_vision содержит camera_detector_node. Упрощенная логика: фиксированная модель загружается из папки models внутри пакета, входной топик /image_raw, выходной топик /image_processed, compressed-копия /image_processed/compressed и отдельный /detections с JSON-описанием найденных объектов. Это сделано так, чтобы видео можно было смотреть с рамками, а детекции использовать отдельно в логике робота.

Параметр	Назначение
enabled	Включает/выключает обработку. В вебе переключатель должен управлять этим параметром.
input_topic	Обычно /image_raw.
output_topic	Обычно /image_processed.
detections_topic	Обычно /detections.
confidence_threshold / nms_threshold	Фильтрация слабых и пересекающихся bbox.

Карты

В корне есть папка maps. В ней лежат карты стандартного формата Nav2: YAML с resolution/origin и PGM/PNG изображением occupancy grid. В веб-визуализации карта может отображаться полупрозрачно, а одометрия накладывается так, чтобы масштаб совпадал с resolution карты: например resolution=0.05 означает 5 см на клетку.

Файл	Назначение
maps/README.md	Карта или описание карты.
maps/test_map.pgm	Карта или описание карты.
maps/test_map.yaml	Карта или описание карты.

9. Навигация, SLAM и локализация

Nav2

rover_navigation содержит nav2_params.yaml для Nav2. Конфиг использует map/odom/base_link frames, AMCL с OmniMotionModel для Mecanum-платформы, DWB local planner, global/local costmaps и /scan как источник препятствий.

Компонент	Настройка
AMCL	robot_model_type: nav2_amcl::OmniMotionModel
Planner	NavFnPlanner
Controller	DWBLocalPlanner
Costmaps	global_costmap и local_costmap, obstacle/voxel/inflation layers
Sensor	/scan LaserScan
Map resolution	обычно 0.05 м/клетку

SLAM Toolbox

mapping.launch.py поднимает SLAM Toolbox для построения карты по /scan и /odom. Полученные карты можно сохранять в maps/ и затем использовать в Nav2 и веб-визуализации.

Локализация

rover_localization хранит EKF-конфиги: wheel-only и вариант с IMU. Wheel odometry публикует /odom из энкодеров. EKF может объединять wheel odometry и IMU для более устойчивого /odometry/filtered в будущей конфигурации.

10. Голосовой модуль Waveshare

Новый голосовой стек рассчитан на Waveshare ESP32-S3 Audio Board. На сам модуль заливается `firmware`, который получает звук с микрофона и передает его на Raspberry Pi. На Raspberry Pi пакет `rover_waveshare_audio` читает `serial`-поток, собирает аудиофрагменты и распознает речь через Whisper.

Часть	Что делает
Firmware ESP32-S3	Работает как аудиоисточник: I2S microphone -> serial packets/WAV stream.
<code>rover_waveshare_audio</code>	Принимает звук, запускает Whisper, публикует текст.
Конфиг	<code>serial_port</code> , <code>baudrate</code> , <code>whisper_model</code> , <code>language</code> , <code>output_text_topic</code> .
Выход	<code>/voice/text</code> и статусный топик.

Модель Whisper настраивается в конфиге: `tiny/base/small/medium/large`. Для Raspberry Pi обычно разумно начинать с `base` или `small`, потому что `large` потребует значительно больше CPU/RAM и времени обработки.

11. Конфиги и важные файлы

Главные конфиги

Файл	Что важно
src/rover_bringup/config/rover.yaml	Геометрия робота, base_driver, IMU, wheel odometry, общие параметры.
src/rover_bringup/config/peripherals.yaml	Лидар, камера, vision, LED, Octoliner, Waveshare audio.
src/rover_bringup/config/ui.yaml	Связка UI launch, экран и веб-компоненты.
src/ui/rover_web/config/web.yaml	Порты веба/терминала/rosboard, maps_root, hackathon_files_root, servo_enabled, topics.
src/ui/rover_display/config/display.yaml	Экран, Wi-Fi, netplan, agent/battery topics, консоль.
src/rover_navigation/config/nav2_params.yaml	Nav2 параметры.
src/rover_navigation/config/slam_toolbox_params.yaml	SLAM Toolbox параметры.
src/rover_localization/config/*.yaml	EKF wheel-only и IMU конфигурации.

Найденные config-файлы

Файл	Тип
src/peripherals/rover_base_driver/config/motor_board.yaml	config
src/peripherals/rover_camera/config/camera.yaml	config
src/peripherals/rover_led_strip/config/led_strip.yaml	config
src/peripherals/rover_octoliner/config/octoliner.yaml	config
src/peripherals/rover_waveshare_audio/config/waveshare_audio.yaml	config
src/rover_bringup/config/peripherals.yaml	config
src/rover_bringup/config/rover.yaml	config
src/rover_bringup/config/twist_mux.yaml	config
src/rover_bringup/config/ui.yaml	config
src/rover_localization/config/ekf_wheel_only.yaml	config
src/rover_localization/config/ekf_with_imu.yaml	config
src/rover_navigation/config/nav2_params.yaml	config
src/rover_navigation/config/slam_toolbox_params.yaml	config
src/rover_teleop/config/teleop.yaml	config
src/rover_vision/config/vision.yaml	config
src/ui/rover_display/config/display.yaml	config
src/ui/rover_web/config/robot_identity.yaml	config
src/ui/rover_web/config/web.yaml	config

Hackathon files

Папка hackathon_files используется веб-страницей Хакатон. В нее можно класть Markdown, HTML и PDF: регламенты соревнований, задания, инструкции, чек-листы и этот документ.

Файл	Назначение
hackathon_files/.DS_Store	Документ/регламент/справочный файл.
hackathon_files/sverk_rover_full_documentation.pdf	Документ/регламент/справочный файл.
hackathon_files/Регламент_Город_дронов_Архипелаг_2026.pdf	Документ/регламент/справочный файл.

12. Проверка, диагностика и типовые проблемы

Быстрые проверки

ROS graph и базовые системные проверки:

```
ros2 node list
ros2 topic list
ros2 service list
ros2 topic echo /cmd_vel
ros2 topic echo /wheel/encoders
ros2 topic echo /odom
```

Камера и нейросеть:

```
ros2 topic hz /image_raw
ros2 topic echo /detections
ros2 service call /usb_camera_node/get_frame rover_interfaces/srv/GetFrame "{}"
```

LED, Octoliner и голос:

```
ros2 topic echo /led_strip/state
ros2 service list | grep led
ros2 topic echo /octoliner/reading
ros2 service call /octoliner/optimize_on_black std_srvs/srv/Trigger "{}"
ros2 topic echo /voice/text
ros2 topic echo /waveshare_audio/status
```

Типовые проблемы

Симптом	Причина/решение
Configured motor_controller path is unavailable	USB-путь изменился. Проверить кабели или заново запустить <code>ros2 run rover_device_manager setup_devices</code> .
cv2 падает из-за NumPy 2.x	Запуск <code>web/camera/vision</code> использует <code>PYTHONNOUSERSITE=1</code> . Для сборки может помочь <code>PYTHONNOUSERSITE=1 colcon build</code> .
Веб не открывается	Проверить <code>http://<ip>:8765</code> , firewall, что <code>web_gateway_node</code> слушает 0.0.0.0, и что IP ровера в той же сети.
Терминал reconnecting	Проверить <code>tytd</code> , shell script <code>rover_terminal_shell.sh</code> , рабочую директорию и права пользователя <code>pi</code> .
LED не светит	Проверить SPI bus/device, MOSI для выбранного SPI, питание ленты, общую землю, <code>led_count=16</code> .
Octoliner не стартует из-за board revision	Использовать текущий драйвер без старых Raspberry Pi board detection зависимостей; проверить I2C bus/address.
Whisper не загружается	Проверить <code>openai-whisper/torch</code> , <code>setuptools/coverage compatibility</code> , <code>serial /dev/waveshare_audio</code> .
Карта в вебе не совпадает с роботом	Визуализация использует <code>/odom</code> как координаты карты. Стартовать из <code>origin</code> или учитывать <code>map->odom transform</code> в будущей версии.

Минимальный чек-лист перед хакатоном

- Собрать workspace и `source install/setup.zsh`.
- Проверить stable device paths через `rover_device_manager`.
- Запустить `ui.launch.py` и открыть веб с ноутбука.
- Проверить `/cmd_vel` без поднятых колес только на безопасной скорости; первый тест лучше делать с колесами в воздухе.
- Проверить `/image_raw`, `/detections`, `/scan`, `/odom`, `/led_strip/state`, `/octoliner/reading`, `/voice/text`.
- Положить регламенты и этот PDF в `hackathon_files`, карты для веб-визуализации в `maps/`.